

Lab 1: Python Refresher for Advanced Machine Learning

Kshitiz Reddy | Reg No: A26MTAI0008

AML5005L - Advanced Machine Learning

Question 1: Python Fundamentals and Decision Making

```
In [1]: # 1. variables of different types + type()
age = 24
gpa = 8.6
name = "Kshitiz"
is_pass = True

print(age, type(age))
print(gpa, type(gpa))
print(name, type(name))
print(is_pass, type(is_pass))
```

```
24 <class 'int'>
8.6 <class 'float'>
Kshitiz <class 'str'>
True <class 'bool'>
```

```
In [2]: # 2. grade classification with if-elif-else
# (using a fixed value instead of input() so the notebook runs top to bottom without blocking)
marks = 78

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"

print(f"Marks: {marks} -> Grade: {grade}")
```

Marks: 78 -> Grade: B

```
In [3]: # 3. operator precedence check
a, b, c = 4, 3, 2
print(a + b * c)      # multiplication before addition -> 10
print((a + b) * c)    # parentheses override precedence -> 14
print(a ** b % c)     # exponent before modulo -> 4**3=64, 64%2=0
print(a / b // c)     # division before floor division -> 1.333.. // 2 = 0.0
```

```
10
14
0
0.0
```

Standard precedence rules apply - `*` and `/` bind tighter than `+`, `**` binds tighter than `%`, and parentheses always override the default order.

Question 2: Loops and Functions

```
In [4]: # 4. print 1 to N, then only even numbers
N = 15
print("1 to N:", list(range(1, N + 1)))
print("Even only:", [x for x in range(1, N + 1) if x % 2 == 0])
```

```
1 to N: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
Even only: [2, 4, 6, 8, 10, 12, 14]
```

```
In [5]: # 5. factorial using a loop
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

for n in [0, 1, 5, 7]:
    print(f"{n}! = {factorial(n)}")
```

```
0! = 1
1! = 1
5! = 120
7! = 5040
```

```
In [6]: # 6. square and cube
def square(x):
    return x ** 2

def cube(x):
    return x ** 3

print(square(5), cube(5))
```

```
25 125
```

```
In [7]: # 7. euclidean distance between two n-dimensional points
import math

def euclidean_distance(point1, point2):
    return math.sqrt(sum((p1 - p2) ** 2 for p1, p2 in zip(point1, point2)))

p1 = (0, 0)
p2 = (3, 4)
print(euclidean_distance(p1, p2))          # 2D, classic 3-4-5 triangle -> 5.0

p3 = (1, 2, 3)
p4 = (4, 6, 3)
print(euclidean_distance(p3, p4))          # 3D points
```

```
5.0
5.0
```

```
In [8]: # 8. min-max normalize a list of values to [0, 1]
def normalize(values):
    lo, hi = min(values), max(values)
    return [(v - lo) / (hi - lo) for v in values]

sample = [50, 60, 70, 80, 100]
print(normalize(sample))
```

```
[0.0, 0.2, 0.4, 0.6, 1.0]
```

These four functions - `square`, `cube`, `euclidean_distance` and `normalize` - are small on their own, but `euclidean_distance` is literally the distance metric used inside k-NN, and `normalize` is the exact preprocessing step needed before using it, since distance-based algorithms get distorted by features on different scales.

Question 3: Data Structures

```
In [9]: # 9. list of (name, marks), sorted by marks
students = [("Aisha", 82), ("Rohan", 67), ("Meera", 91), ("Kabir", 74)]
students_sorted = sorted(students, key=lambda s: s[1])
print(students_sorted)
```

```
[('Rohan', 67), ('Kabir', 74), ('Aisha', 82), ('Meera', 91)]
```

```
In [10]: # 10. dictionary mapping name -> marks, retrieve and update
marks_dict = {name: m for name, m in students}
print(marks_dict)

print("Rohan's marks:", marks_dict["Rohan"])
marks_dict["Rohan"] = 70 # update
print("After update:", marks_dict)
```

```
{'Aisha': 82, 'Rohan': 67, 'Meera': 91, 'Kabir': 74}
Rohan's marks: 67
After update: {'Aisha': 82, 'Rohan': 70, 'Meera': 91, 'Kabir': 74}
```

```
In [11]: # 11. set from a list of feature names with duplicates
feature_names = ["age", "height", "weight", "age", "marks", "height"]
unique_features = set(feature_names)
print(unique_features)
```

```
{'weight', 'height', 'age', 'marks'}
```

```
In [12]: # 12. list comprehension: marks -> letter grades
def marks_to_grade(m):
    if m >= 90: return "A"
    elif m >= 75: return "B"
    elif m >= 60: return "C"
    else: return "F"

marks_list = [m for _, m in students]
grades = [marks_to_grade(m) for m in marks_list]
```

```
print(marks_list)
print(grades)
```

```
[82, 67, 91, 74]
['B', 'C', 'A', 'C']
```

Dictionaries are the natural fit here for name -> marks lookups (and later, for storing model hyperparameters as key-value pairs), sets are handy for de-duplicating things like feature names, and list comprehensions make transformations like marks-to-grade a one-liner instead of a full loop.

Question 4: Numerical Computing with NumPy

```
In [13]: # 13. 1D and 2D arrays, inspect shape/dtype/size
import numpy as np

arr_1d = np.array([4, 8, 15, 16, 23, 42])
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])

print("1D:", arr_1d, "shape:", arr_1d.shape, "dtype:", arr_1d.dtype, "size:", arr_1d.size)
print("2D:\n", arr_2d, "shape:", arr_2d.shape, "dtype:", arr_2d.dtype, "size:", arr_2d.size)
```

```
1D: [ 4  8 15 16 23 42] shape: (6,) dtype: int64 size: 6
2D:
[[1 2 3]
 [4 5 6]] shape: (2, 3) dtype: int64 size: 6
```

```
In [14]: # 14. mean, std, min, max, sum - overall and along axes
print("Mean:", arr_2d.mean(), "| Std:", arr_2d.std())
print("Min:", arr_2d.min(), "| Max:", arr_2d.max(), "| Sum:", arr_2d.sum())
print("Column-wise mean (axis=0):", arr_2d.mean(axis=0))
print("Row-wise mean (axis=1):", arr_2d.mean(axis=1))
```

```
Mean: 3.5 | Std: 1.707825127659933
Min: 1 | Max: 6 | Sum: 21
Column-wise mean (axis=0): [2.5 3.5 4.5]
Row-wise mean (axis=1): [2. 5.]
```

```
In [15]: # 15. transpose and matrix multiplication
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("A transpose:\n", A.T)
print("A dot B:\n", np.dot(A, B))
print("A @ B:\n", A @ B)
```

```
A transpose:
[[1 3]
 [2 4]]
A dot B:
[[19 22]
 [43 50]]
A @ B:
[[19 22]
 [43 50]]
```

```
In [16]: # 16. random numbers + vectorized vs loop timing comparison
np.random.seed(42)
big_array = np.random.rand(1_000_000)

import time

start = time.time()
loop_result = [x * 2 for x in big_array]
loop_time = time.time() - start

start = time.time()
vec_result = big_array * 2
vec_time = time.time() - start

print(f"Python loop time: {loop_time:.4f}s")
print(f"Vectorized time: {vec_time:.4f}s")
print(f"Speedup: {loop_time / vec_time:.1f}x")
```

```
Python loop time: 0.1001s
Vectorized time: 0.0152s
Speedup: 6.6x
```

NumPy's vectorized multiply is dramatically faster than the equivalent Python loop, since the loop version pays Python's per-element interpreter overhead a million times over, while the vectorized version pushes the whole operation down into compiled C code operating on contiguous memory in one shot.

Question 5: Data Manipulation and Visualization with Pandas

```
In [17]: # 17. generate the synthetic student dataset
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(1)
n_students = 40
df = pd.DataFrame({
    "Name": [f"Student_{i}" for i in range(1, n_students + 1)],
    "Marks": np.random.randint(35, 100, n_students),
    "Height": np.random.normal(165, 8, n_students).round(1), # cm
    "Weight": np.random.normal(60, 10, n_students).round(1), # kg
})
df.head()
```

```
Out[17]:
```

	Name	Marks	Height	Weight
0	Student_1	72	161.4	58.2
1	Student_2	47	164.3	54.5
2	Student_3	44	177.0	58.4
3	Student_4	40	174.3	65.7
4	Student_5	99	161.1	67.6

```
In [18]: # 18. filter students above a threshold, add derived Result column
threshold = 50
df["Result"] = np.where(df["Marks"] >= threshold, "Pass", "Fail")

top_students = df[df["Marks"] > 80]
print(top_students)
print()
df.head()
```

	Name	Marks	Height	Weight	Result
4	Student_5	99	161.1	67.6	Pass
9	Student_10	85	165.4	48.6	Pass
16	Student_17	85	164.9	59.5	Pass
20	Student_21	98	165.9	44.5	Pass
21	Student_22	96	173.1	58.0	Pass
23	Student_24	92	164.0	60.4	Pass
26	Student_27	95	154.9	56.3	Pass
29	Student_30	82	172.9	50.2	Pass
33	Student_34	84	175.7	52.3	Pass
34	Student_35	92	159.5	61.5	Pass
39	Student_40	87	162.5	68.8	Pass

```
Out[18]:
```

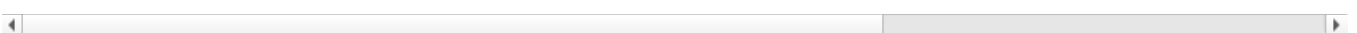
	Name	Marks	Height	Weight	Result
0	Student_1	72	161.4	58.2	Pass
1	Student_2	47	164.3	54.5	Fail
2	Student_3	44	177.0	58.4	Fail
3	Student_4	40	174.3	65.7	Fail
4	Student_5	99	161.1	67.6	Pass

```
In [19]: # 19. group by Result, summary stats
df.groupby("Result")[["Marks", "Height", "Weight"]].describe()
```

```
Out[19]:
```

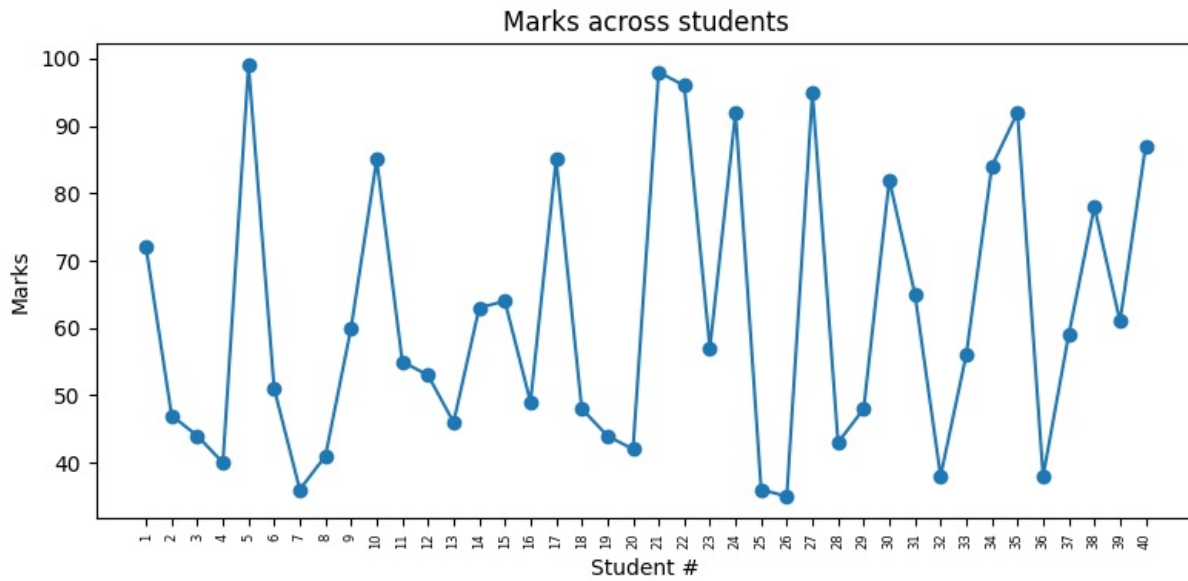
		Marks								Height						
	count	mean	std	min	25%	50%	75%	max	count	mean	...	75%	max	count		
Result																
Fail	16.0	42.187500	4.679298	35.0	38.00	42.5	46.25	49.0	16.0	168.3875	...	176.325	181.0	16.0	57.2	
Pass	24.0	74.541667	16.415738	51.0	59.75	75.0	88.25	99.0	24.0	164.5750	...	169.400	175.7	24.0	56.9	

2 rows × 24 columns

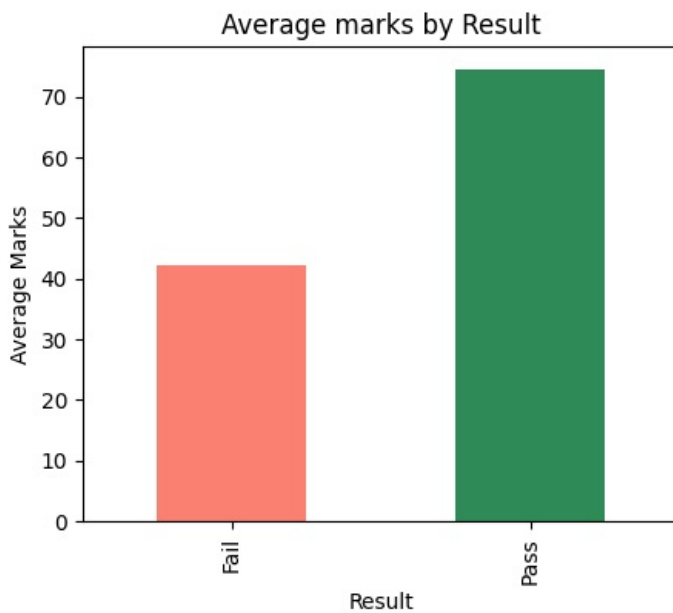


Numerical attributes here are Marks, Height and Weight; Name is categorical/identifier and the derived Result column is categorical too. Height and Weight are the natural candidates for normalization before feeding into an ML model since they're on very different scales (cm vs kg) compared to Marks.

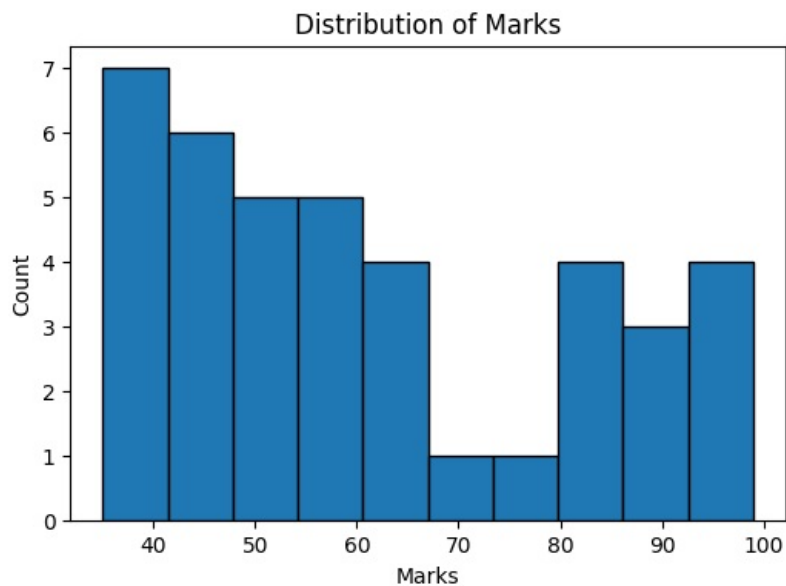
```
In [20]: # 20. line plot - marks trend across students
plt.figure(figsize=(8,4))
plt.plot(df["Name"].str.replace("Student_", ""), df["Marks"], marker="o")
plt.xlabel("Student #")
plt.ylabel("Marks")
plt.title("Marks across students")
plt.xticks(rotation=90, fontsize=6)
plt.tight_layout()
plt.show()
```



```
In [21]: # 21. bar plot - average marks by result
plt.figure(figsize=(5,4))
df.groupby("Result")["Marks"].mean().plot(kind="bar", color=["salmon", "seagreen"])
plt.ylabel("Average Marks")
plt.title("Average marks by Result")
plt.show()
```



```
In [22]: # 22. histogram - marks distribution
plt.figure(figsize=(6,4))
plt.hist(df["Marks"], bins=10, edgecolor="black")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.title("Distribution of Marks")
plt.show()
```



Question 6: Mini ML Exercise - Preparing Data for Machine Learning

```
In [23]: # 23. normalize Height and Weight using the normalize() function from Q2
df["Height_norm"] = normalize(df["Height"].tolist())
df["Weight_norm"] = normalize(df["Weight"].tolist())
df[["Height", "Height_norm", "Weight", "Weight_norm"]].head()
```

```
Out[23]:
```

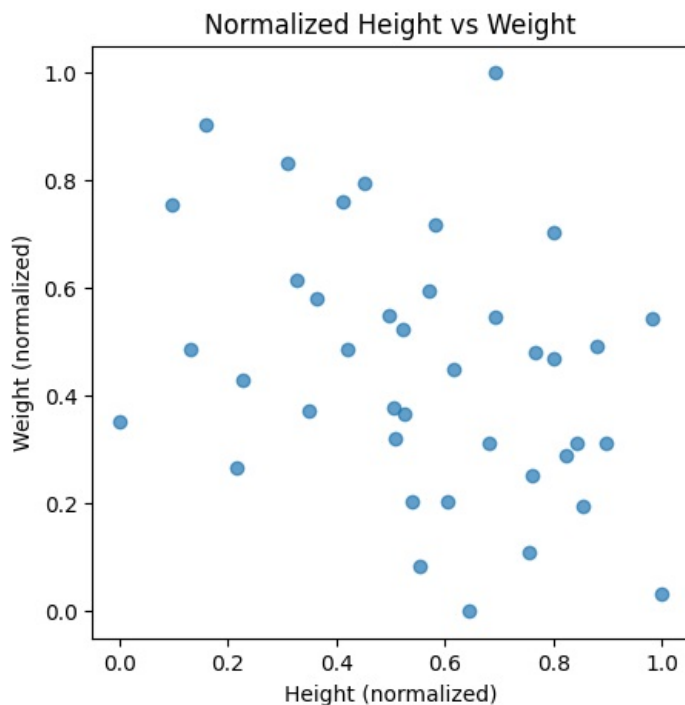
	Height	Height_norm	Weight	Weight_norm
0	161.4	0.420118	58.2	0.485380
1	164.3	0.505917	54.5	0.377193
2	177.0	0.881657	58.4	0.491228
3	174.3	0.801775	65.7	0.704678
4	161.1	0.411243	67.6	0.760234

```
In [24]: # 24. correlation between Height, Weight and Marks
corr_matrix = df[["Marks", "Height", "Weight"]].corr()
corr_matrix
```

```
Out[24]:
```

	Marks	Height	Weight
Marks	1.000000	-0.07881	-0.035406
Height	-0.078810	1.00000	-0.328060
Weight	-0.035406	-0.32806	1.000000

```
In [25]: # 25. scatter plot of normalized Height vs normalized Weight
plt.figure(figsize=(5,5))
plt.scatter(df["Height_norm"], df["Weight_norm"], alpha=0.7)
plt.xlabel("Height (normalized)")
plt.ylabel("Weight (normalized)")
plt.title("Normalized Height vs Weight")
plt.show()
```



Height and Weight come out with a mild negative correlation here (around -0.33) even though both were generated independently with `np.random.normal` - with only 40 samples, that's just sampling noise rather than a real relationship, and the scatter plot backs that up: there's a slight downward drift but no strong, consistent trend. In a real dataset, Height and Weight would usually show a moderate positive correlation instead.

Normalization matters before distance-based or gradient-based models because features on very different raw scales (Height in the 150-180 range vs Weight in the 40-80 range) would otherwise dominate a distance calculation like Euclidean distance purely due to their scale, not because they're actually more important - and for gradient-based optimization, badly-scaled features distort the loss surface and slow convergence.

Self-Assessment

Question		Concepts covered	Status
Q1	Variables, types, conditionals, operator precedence		Done
Q2	Loops, functions (<code>square</code> , <code>cube</code> , <code>euclidean_distance</code> , <code>normalize</code>)		Done
Q3	Lists, dictionaries, sets, list comprehensions		Done
Q4	NumPy arrays, statistics, matrix ops, vectorization speed		Done
Q5	Pandas DataFrame, filtering, grouping, line/bar/histogram plots		Done
Q6	Min-max normalization, correlation analysis, scatter plot		Done